Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....5
1.1 Описание входных данных.....7
1.2 Описание выходных данных.....7
2 МЕТОД РЕШЕНИЯ.....9
3 ОПИСАНИЕ АЛГОРИТМОВ.....12
3.1 Алгоритм метода set_private_val класса Sensor.....12
3.2 Алгоритм конструктора класса Sensor.....12
3.3 Алгоритм метода set_values класса Sensor.....13
3.4 Алгоритм метода print класса Sensor.....13
3.5 Алгоритм конструктора класса SmartSensor.....14
3.6 Алгоритм метода set_values класса SmartSensor.....14
3.7 Алгоритм метода print класса SmartSensor.....15
3.8 Алгоритм функции main.....15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....17
5 КОД ПРОГРАММЫ.....21
5.1 Файл main.cpp.....21
5.2 Файл Sensor.cpp.....22
5.3 Файл Sensor.h.....22
5.4 Файл SmartSensor.cpp.....23
5.5 Файл SmartSensor.h.....23
6 ТЕСТИРОВАНИЕ.....25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....26

1 ПОСТАНОВКА ЗАДАЧИ

В системе мониторинга используется **базовый датчик**, который хранит внутренний калибровочный параметр и измеряемое значение. На основе базового датчика разрабатывается **умный датчик**, который наследует свойства базового датчика, но имеет собственные поля и методы с теми же именами.

Требуется разработать систему, которая

- демонстрирует создание объекта производного класса;
- демонстрирует сохранение доступа к открытым элементам базового класса;
- демонстрирует сокрытие одноимённых элементов базового класса;
- демонстрирует однозначное обращение к элементам базового и производного классов.

Базовый класс должен содержать:

- в закрытом доступе:
  - одно свойство целого типа;
  - метод с одним параметром, который устанавливает значение закрытого свойства равным утроенному значению параметра.
- в открытом доступе:
  - одно свойство целого типа;
  - параметризированный конструктор с двумя параметрами(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
  - метод с двумя параметрами для изменения значений свойств(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
  - метод вывода значений свойств в формате "Sensor: <закрытое

свойство> <открытое свойство>"

Производный класс должен наследоваться от базового класса таким образом, чтобы к открытым элементам исходного класса сохранить открытый доступ и у него:

- в закрытом доступе:
  - одно свойство целого типа с тем же именем, что и закрытое свойство базового класса.
- в открытом доступе:
  - одно свойство целого типа с тем же именем, что и открытое свойство базового класса;
  - параметризированный конструктор (1 параметр - установка закрытого свойства, 2 параметр - установка открытого свойства);
  - метод изменения значений свойств с двумя параметрами;
  - метод вывода значений свойств в формате: "SmartSensor: <закрытое свойство производного класса> <открытое свойство производного класса>"

Алгоритм работы программы

1. Объявить две целочисленные переменные.
2. Ввести значения двух переменных.
3. Создать объект производного класса.
4. Вывести значения свойств базовой части объекта.
5. Вывести значения свойств производной части объекта.
6. Ввести одно целое число.
7. Присвоить это значение открытому свойству производного класса.
8. Присвоить это значение открытому свойству базового класса.
9. Вывести значения свойств производного объекта.
10. Вывести значения свойств базового объекта.

11. Ввести два целых числа a и b.
12. Если первое число больше нуля:
    12.1. вызвать метод установки свойств производного объекта с параметрами a+1 и b+1;
    12.2. вызвать метод установки свойств базового объекта с параметрами a-1 и b-1;
    12.3. вывести значения производного объекта;
    12.4. вывести значения базового объекта.
13. Иначе:
    13.1. вызвать метод установки свойств базового объекта с параметрами a+1 и b+1;
    13.2. вызвать метод установки свойств производного объекта с параметрами a-1 и b-1;
    13.3. вывести значения базового объекта;
    13.4. вывести значения производного объекта.
14. Завершить работу программы.

1.1 Описание входных данных

Пример входных данных:

```
3 4
5
2 3
```

1.2 Описание выходных данных

Пример выходных данных:

```
Sensor: 9 4
SmartSensor: 3 4
SmartSensor: 3 5
```

```
Sensor: 9 5
SmartSensor: 3 4
Sensor: 3 2
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект obj класса SmartSensor предназначен для демонстрации наследования, сокрытия имён и доступа к базовым элементам через оператор разрешения области видимости;
- функция main для основная функция программы.

Класс Sensor:

- свойства/поля:
  - поле для хранения утроенного значения параметра:
    - наименование — private_val;
    - тип — int;
    - модификатор доступа — private;
  - поле для хранения и приямого изменения значения:
    - наименование — public_val;
    - тип — int;
    - модификатор доступа — public;
- функционал:
  - метод set_private_val — устанавливает значение private_val равным утроенному значению параметра val;
  - метод Sensor — конструктор класса. Первый параметр priv используется для установки закрытого свойства через метод set_private_val, второй парметр pub для установки открытого свойства public_val;
  - метод set_values — для изменения значений свойств датчика. Первый параметр priv передается в set_private_val, второй параметр pub присваивается в public_val;

- метод print — вывод значений свойств датчика в формате: "Sensor: <private_val> <public_val>".

Класс SmartSensor:

- свойства/поля:
  - поле private_val в классе SmartSenor, которая скрывает одноимённое поле из родительского класса:
    - наименование — private_val;
    - тип — int;
    - модификатор доступа — private;
  - поле public_val в классе SmartSenor, которая скрывает одноимённое поле из родительского класса:
    - наименование — public_val;
    - тип — int;
    - модификатор доступа — public;
- функционал:
  - метод SmartSensor — конструктор дочернего класса. Вызывает конструтор родительского класса Sensor с переданными параметрами priv и pub, затем инициализирует собственные поля private_val и public_val значениями этих же параметров;
  - метод set_values — переопределённый метод изменения значений свойств умного датчика. Присваивает переданные параметры priv и pub собственными полям private_val и public_val;
  - метод print — переопределённый метод вывода значений свойств умного датчика. Вывод в формате "SmartSensor: <private_val> <public_val>", используя только собсвтенные поля класса.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Sensor			Родительский класс датчика	
		SmartSensor	public		2
2	SmartSensor			Дочерний класс, демонстрирующий сокрытие одноимённых элементов родительского класса.	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода set_private_val класса Sensor

Функционал: устанавливает значение private_val равным утроенному значению параметра val.

Параметры: целочисленный val.

Возвращаемое значение: void, метод не возращает значение.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода set_private_val класса Sensor

№	Предикат	Действия	№ перехода
1		private_val = val * 3	∅

3.2 Алгоритм конструктора класса Sensor

Функционал: конструктор класса. Первый параметр priv используется для установки закрытого свойства через метод set_private_val, второй парметр pub для установки открытого свойства public_val.

Параметры: целочисленый priv, целочисленный pub.

Алгоритм конструктора представлен в таблице 3.

Таблица 3 – Алгоритм конструктора класса Sensor

№	Предикат	Действия	№ перехода
1		установка закрытого свойства priv через метод set_private_val	2

№	Предикат	Действия	№ перехода
2		прямая установка открытого свойства public_val = pub	∅

3.3 Алгоритм метода set_values класса Sensor

Функционал: для изменения значений свойств датчика. Первый параметр priv передается в set_private_val, второй параметр pub присваивается в public_val.

Параметры: целочисленный priv, целочисленный pub.

Возвращаемое значение: void, метод ничего не возращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода set_values класса Sensor

№	Предикат	Действия	№ перехода
1		Вызов метода set_private_val для утроения и записи значения priv	2
2		Прямая установка открытого свойства public_val = pub	∅

3.4 Алгоритм метода print класса Sensor

Функционал: Вывод значений свойств датчика в формате: "Sensor: <private_val> <public_val>".

Параметры: нет.

Возвращаемое значение: void, метод только выводит данные в консоль.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода print класса Sensor

№	Предикат	Действия	№ перехода
1		Вывод свойств в консоль в формате: "Sensor: <private_val> <public_val>	∅

3.5 Алгоритм конструктора класса SmartSensor

Функционал: Конструктор дочернего класса. Вызывает конструтор родительского класса Sensor с переданными параметрами priv и pub, затем инициализирует собственные поля private_val и public_val значениями этих же параметров.

Параметры: целочисленный priv, целочисленный pub.

Алгоритм конструктора представлен в таблице 6.

Таблица 6 – Алгоритм конструктора класса SmartSensor

№	Предикат	Действия	№ перехода
1		Вызов конструтора родительского класса Sensor(priv, pub)	2
2		Инициализация собственного закрытого поля private_val равным параметру priv	3
3		Инициализация собственного открытого поля public_val равным параметру pub	∅

3.6 Алгоритм метода set_values класса SmartSensor

Функционал: Переопределённый метод изменения значений свойств умного датчика. Присваивает переданные параметры priv и pub собственными полям private_val и public_val.

Параметры: целочисленный priv, целочисленный pub.

Возвращаемое значение: void, метод ничего не возращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода set_values класса SmartSensor

№	Предикат	Действия	№ перехода
1		Установка значения собственного скрытого поля private_val равному переданному параметру priv	2

№	Предикат	Действия	№ перехода
2		Установка значения собственного открытого поля public_val равному переданному параметру pub	∅

3.7 Алгоритм метода print класса SmartSensor

Функционал: Переопределённый метод вывода значений свойств умного датчика. Вывод в формате "SmartSensor: <private_val> <public_val>", используя только собсвтенные поля класса.

Параметры: нет.

Возвращаемое значение: void, метод только выводит данные в консоль.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода print класса SmartSensor

№	Предикат	Действия	№ перехода
1		Вывод значений собственных свойств дочернего класса в консоль в формате: "SmartSensor: <private_val> <public_val>	∅

3.8 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: целочисленный, код возврата.

Алгоритм функции представлен в таблице 9.

Таблица 9 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		Объявление переменных int val1, val2	2
2		Ввод значений в val1 и val2 через std::cin	3

№	Предикат	Действия	№ перехода
3		Создание объекта SmartSensor obj(val1, val2)	4
4		Вызов метода родительского класса obj.Sensor::print()	5
5		Вызов метода дочернего класса obj.print()	6
6		Объявление переменной int single_val	7
7		Ввод значения в single_val через std::cin	8
8		Присваивание obj.public_val = single_val	9
9		Присваивание obj.Sensor::public_val = single_val	10
10		Вызов метода obj.print()	11
11		Вызов метода obj.Sensor::print()	12
12		Объявление переменных int a, b	13
13		Ввод значений в a и b через std::cin	14
14	a > 0	Вызов obj.set_values(a+1, b+1)	15
			18
15		Вызов Sensor::set_values(a-1, b-1)	16
16		Вызов метода obj.print()	17
17		Вызов метода obj.Sensor::print()	22
18		Вызов obj.Sensor::set_values(a+1, b+1)	19
19		Вызов obj.set_values(a-1, b-1)	20
20		Вызов метода obj.Sensor::print()	21
21		Вызов метода obj.print()	22
22		Возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-4.

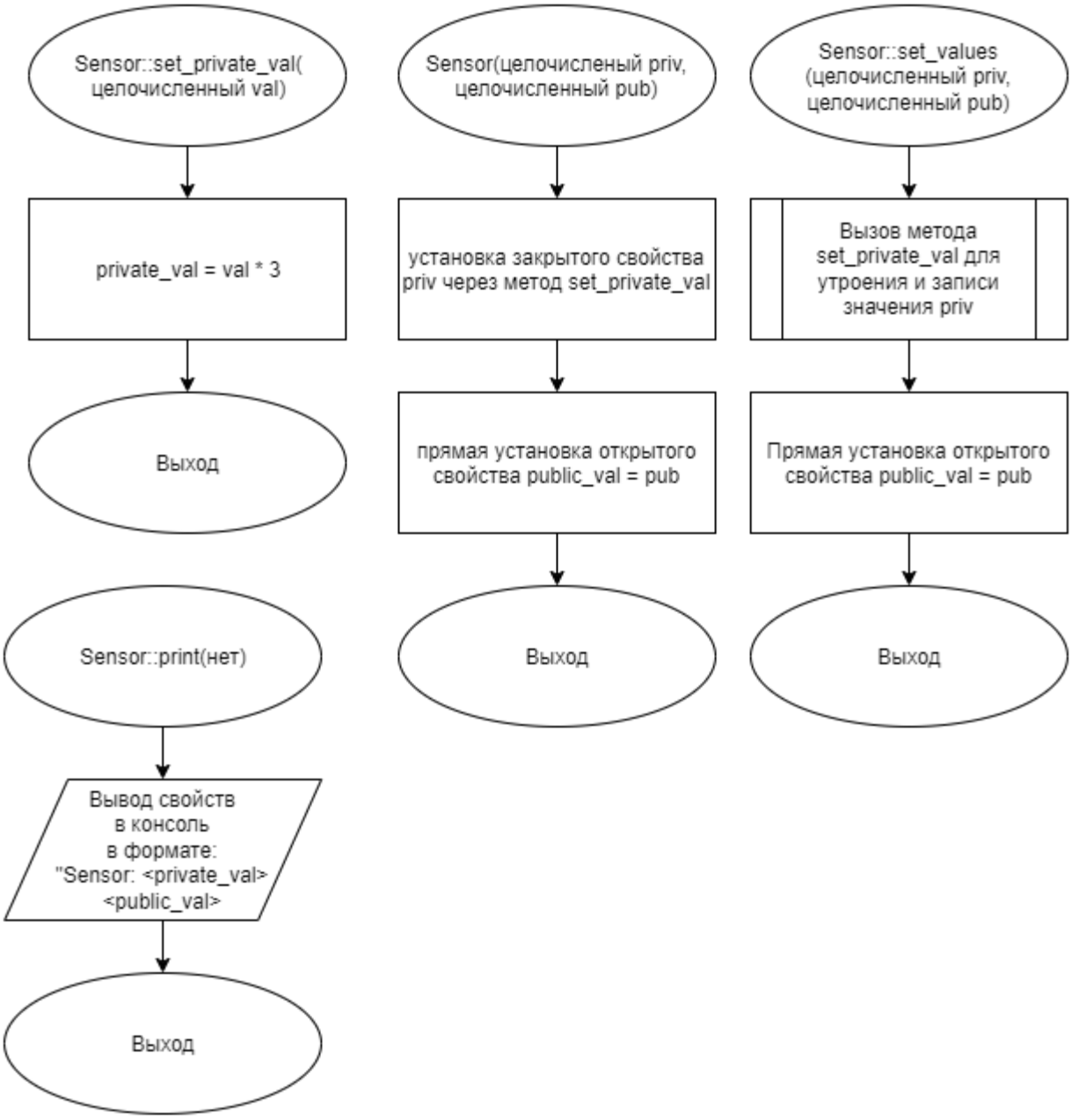

Рисунок 1 – Блок-схема алгоритма

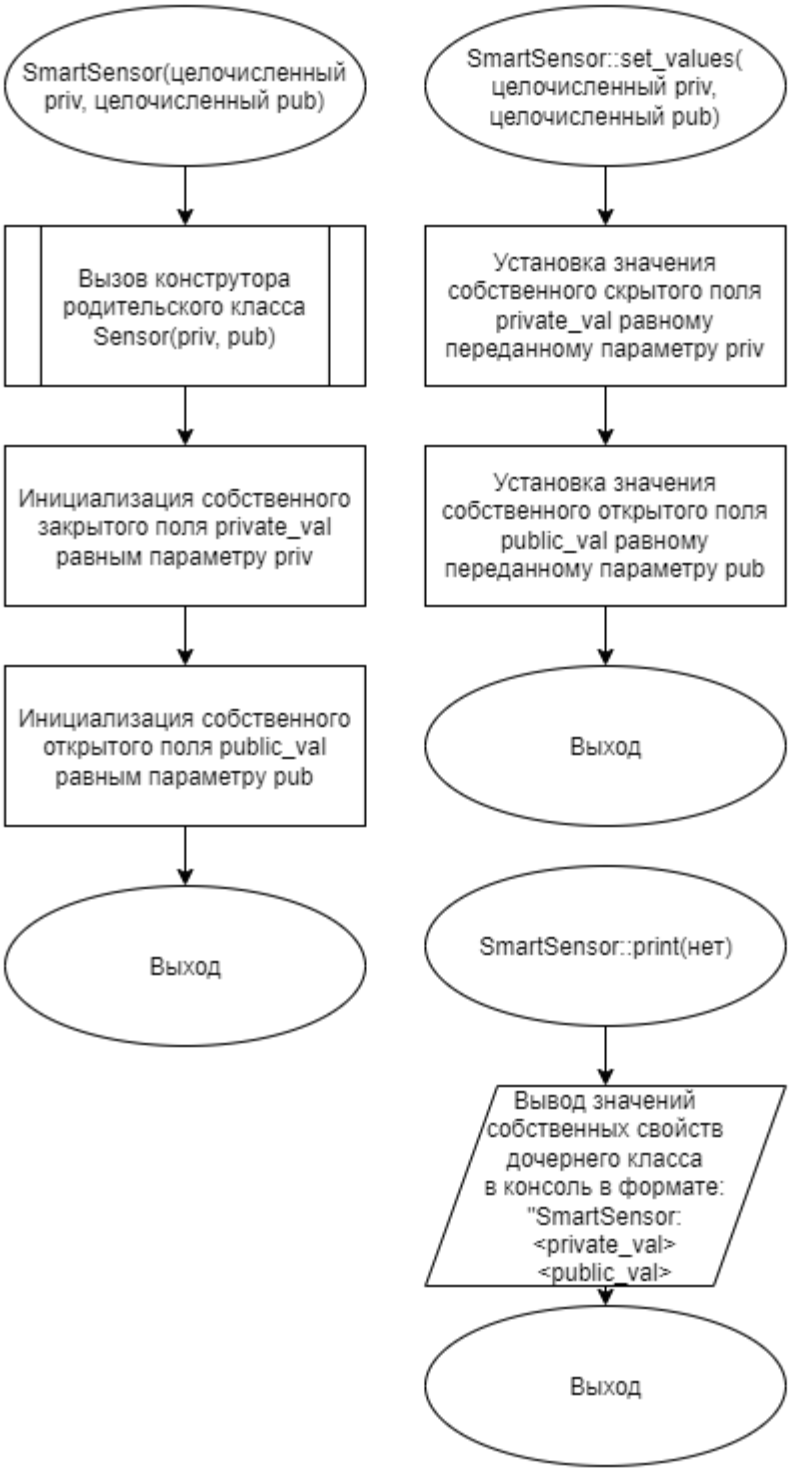

Рисунок 2 – Блок-схема алгоритма

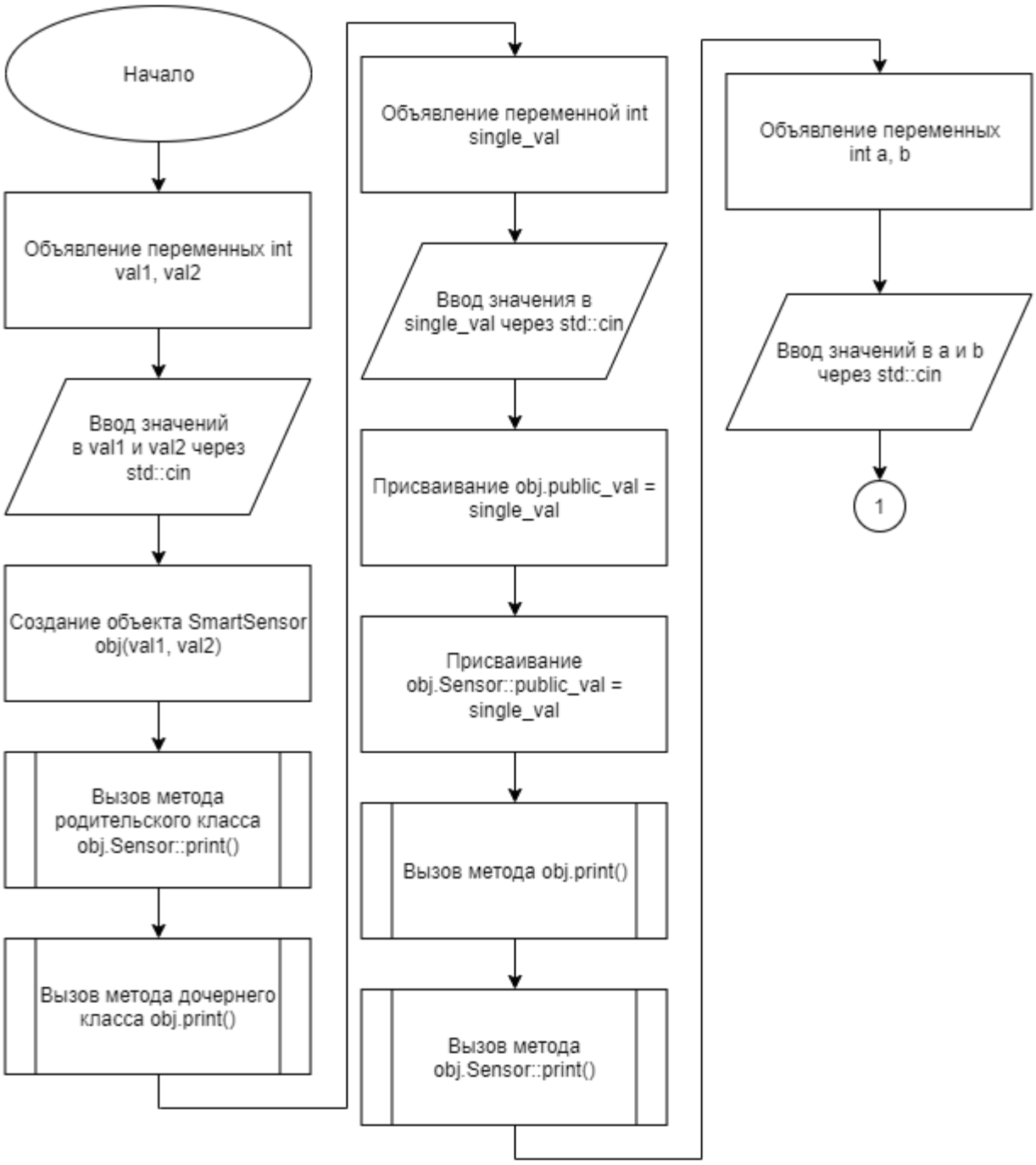

Рисунок 3 – Блок-схема алгоритма

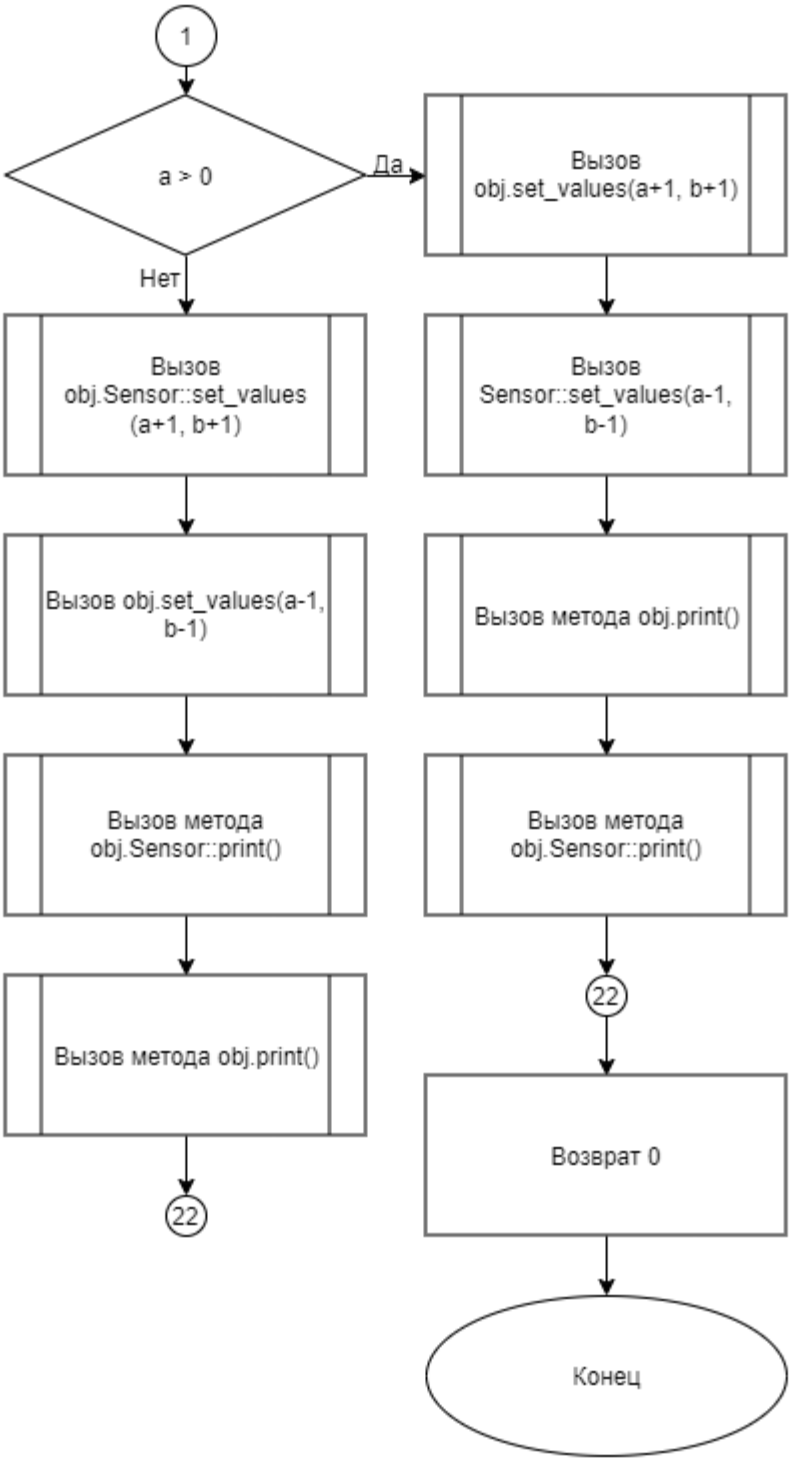

Рисунок 4 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <iostream>
#include "SmartSensor.h"

using namespace std;

int main()
{
   int val1, val2;
   cin >> val1 >> val2;

   SmartSensor obj(val1, val2);

   obj.Sensor::print();
   obj.print();


   int single_val;
   cin >> single_val;

   obj.public_val = single_val;
   obj.Sensor::public_val = single_val;

   obj.print();
   obj.Sensor::print();

   int a, b;
   cin >> a >> b;

   if(a > 0){
      obj.set_values(a + 1, b + 1);
      obj.Sensor::set_values(a - 1, b - 1);
      obj.print();
      obj.Sensor::print();
   }
   else{
      obj.Sensor::set_values(a + 1, b + 1);
      obj.set_values(a - 1, b - 1);
      obj.Sensor::print();
      obj.print();
```

```
        }

        return(0);
    }
```

5.2 Файл Sensor.cpp

Листинг 2 – Sensor.cpp

```
    #include "Sensor.h"
    #include <iostream>

    void Sensor::set_private_val(int val){
        private_val = val * 3;
    }

    Sensor::Sensor(int priv, int pub){
        set_private_val(priv);
        public_val = pub;
    }

    void Sensor::set_values(int priv, int pub){
        set_private_val(priv);
        public_val = pub;
    }

    void Sensor::print() const{
        std::cout << "Sensor: " << private_val << " " << public_val << std::endl;
    }
```

5.3 Файл Sensor.h

Листинг 3 – Sensor.h

```
    #ifndef __SENSOR__H
    #define __SENSOR__H

    class Sensor {
    private:
        int private_val;
        void set_private_val(int val);

    public:
        int public_val;
        Sensor(int priv, int pub);
```

```
   void set_values(int priv, int pub);
   void print() const;

};

#endif
```

5.4 Файл SmartSensor.cpp

Листинг 4 – SmartSensor.cpp

```
#include "SmartSensor.h"
#include <iostream>

SmartSensor::SmartSensor(int priv, int pub) : Sensor(priv, pub){
   private_val = priv;
   public_val = pub;
}

void SmartSensor::set_values(int priv, int pub){
   private_val = priv;
   public_val = pub;
}

void SmartSensor::print() const{
   std::cout << "SmartSensor: " << private_val << " " << public_val <<
std::endl;
}
```

5.5 Файл SmartSensor.h

Листинг 5 – SmartSensor.h

```
#ifndef __SMARTSENSOR__H
#define __SMARTSENSOR__H

#include "Sensor.h"

class SmartSensor : public Sensor {
private:
   int private_val;

public:
   int public_val;
   SmartSensor(int priv, int pub);
```

```
        void set_values(int priv, int pub);
        void print() const;

    };

    #endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 10.

Таблица 10 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные

`3 4`
`5`
`2 3`
	`Sensor: 9 4`
	`SmartSensor: 3 4`
	`SmartSensor: 3 5`
	`Sensor: 9 5`
	`SmartSensor: 3 4`
	`Sensor: 3 2`
		`Sensor: 9 4`
		`SmartSensor: 3 4`
		`SmartSensor: 3 5`
		`Sensor: 9 5`
		`SmartSensor: 3 4`
		`Sensor: 3 2`

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).